

Lumen

Implementation Plan

Internal working document for the implementation phase. Six-sprint plan from May 2 to June 15, covering system architecture, build order, and team workload distribution.

TEAM

Diaa Badaha · 1210478

Ahmad Hamdan · 1210241

Omar Husein · 1212738

Contents

- 1. Purpose of This Document**
- 2. Timeline and Operating Constraints**
 - 2.1 Hard constraints we have accepted
 - 2.2 Things we deliberately cut to fit 6.5 weeks
- 3. System Architecture**
 - 3.1 Three Layers
 - 3.2 Data Flow During an Active Task
- 4. Build Plan: Six Sprints**
 - 4.1 Sprint 1: Setup and Vertical Slice (May 5 to May 11)
 - 4.2 Sprint 2: Voice Pipeline (May 12 to May 18)
 - 4.3 Sprint 3: Object Allocation Task (May 19 to June 1, two weeks)
 - 4.4 Sprint 4: Navigation Task (June 2 to June 8)
 - 4.5 Sprint 5: Polish, Trials, and Report (June 9 to June 15)
- 5. Workload Distribution**
 - 5.1 Diaa: Frontend, Audio Pipeline, and Integration
 - 5.2 Ahmad: Vision Pipeline and Object Allocation Task
 - 5.3 Omar: Voice Pipeline, FSM, and Navigation Task
 - 5.4 Shared Responsibilities
- 6. Technical Decisions Locked In**
- 7. Risks and Mitigations**
- 8. Definition of Done for v1**
- 9. Out of Scope (Tracked for Future Work)**

1. Purpose of This Document

This is an internal working document for the Lumen team. It describes how we will build the system between now and the v1 deadline, what we will build in what order, and how the three of us will divide the work. It is not a deliverable for the supervisor; it is a plan we use to keep ourselves aligned.

The introduction report (submitted February 4, 2026) remains the design baseline, but two things have changed since then:

1. **The client device is now a smartphone running a web browser**, not a custom Raspberry Pi wearable. The user opens a URL, grants camera and microphone permissions, and the browser becomes a thin transport layer for sensors and audio playback. The Pi, CSI cable, USB hub, enclosure, and external battery are no longer part of the system.
2. **All perception remains in the cloud**. The browser only captures media, transmits it, and plays back audio responses. It does no inference, no FSM logic, no reasoning. The cloud backend handles everything: YOLOv8n, Whisper, FSM, gTTS.

Where the report and this plan disagree, this plan is authoritative for implementation. The final report will be updated to match what we actually build.

2. Timeline and Operating Constraints

We have **6.5 weeks**, from today (May 2, 2026) to the v1 deadline on **June 15, 2026**. That is half the time we initially scoped, so we are explicit about what we will and will not attempt.

2.1 Hard constraints we have accepted

- **Foreground browser only.** The user must keep the browser tab in the foreground during an active task. If they switch apps or lock the phone, the task ends.
- **Network is required.** No offline mode, no edge inference for v1.
- **Chrome only**, on Android and iOS. Safari and Firefox are out of scope for v1.
- **English-only voice interaction.** Arabic is future work.
- **One concurrent user.** No multi-user concurrency.
- **No empirical user studies with blind participants.** We test among ourselves with blindfolds.

2.2 Things we deliberately cut to fit 6.5 weeks

- **Reconnection logic with task-state recovery.** If the WebSocket drops mid-task, the system falls back to Idle and the user retries. Simple, predictable, and good enough for v1.
- **Hand Pickup Assistance as a separate mode.** The Object Allocation task ends with a “the cup is right in front of you, reach forward” message; we do not implement a fine-grained pickup sub-mode.
- **Custom dataset and model fine-tuning.** We use COCO-pretrained YOLOv8n only. No architectural classes (doors, hallways, stairs).
- **Voice activity detection.** We use push-to-talk for v1. The user holds a button while speaking. Simpler, more reliable, and avoids the echo-cancellation problem entirely.
- **LLM-based guidance phrasing.** Templates only. Variation is nice but not necessary for v1.
- **Performance optimization beyond “it works.”** We do not target specific FPS or latency numbers; we measure what we get.

3. System Architecture

The architecture below replaces Section 4.1 to 4.3 of the introduction report. It is conceptually similar but simpler in execution.

3.1 Three Layers

Client layer: Browser application. A single HTML page with a small JavaScript bundle. Responsibilities: request camera and microphone permissions, capture frames and audio, open a WebSocket to the backend, send media upstream, play back audio responses, render a minimal UI, request a screen wake lock during active tasks.

Transport layer: WebSocket over HTTPS. A single persistent WebSocket connection per session. Binary frames carry JPEG images and audio chunks upstream and MP3 audio responses downstream. JSON text frames carry control messages and FSM state updates in both directions. HTTPS is mandatory because `getUserMedia` does not work over plain HTTP.

Application layer: Python backend. A FastAPI server hosting the WebSocket endpoint, YOLOv8n inference, Whisper STT, the FSM and task manager, a template-based guidance generator, and a gTTS wrapper. The FSM is authoritative on the server. The client maintains no FSM state of its own beyond a “connected / disconnected” indicator.

3.2 Data Flow During an Active Task

1. User taps Start in the browser. Browser requests camera and mic permissions, acquires wake lock, opens WebSocket. Backend creates a session and sends an `idle_ready` message.
2. User holds the push-to-talk button and speaks a command. Frontend records audio while the button is held, then uploads the recording on release.
3. Backend transcribes via Whisper, parses intent, transitions FSM to `ObjectAllocationActive` or `NavigationActive`, and acknowledges with a TTS confirmation.
4. Backend signals the frontend to begin sending frames. Frontend captures camera frames at about 5 FPS, JPEG-encodes them, and pushes them upstream as binary WebSocket messages.
5. Backend runs YOLOv8n on each frame, applies temporal consistency (target detected in 3 of last 5 frames), and emits guidance only when the spatial region or distance category changes.

6. Guidance text is synthesized to MP3 by gTTS and pushed to the frontend, which queues and plays it.
7. User confirms (“got it”), cancels, or the task times out. FSM transitions to `Idle`. Frontend stops sending frames.

4. Build Plan: Six Sprints

We work in one-week sprints. Each sprint produces a working, demonstrable system end-to-end. We do not build a complete frontend, then a complete backend, then connect them; we build vertical slices that get progressively richer. Every Friday we run an end-to-end smoke test together.

4.1 Sprint 1: Setup and Vertical Slice (May 5 to May 11)

Goal: every team member can run the system locally; camera frames go up; placeholder TTS comes back as audio in the browser.

Deliverables:

- Git monorepo with `frontend/`, `backend/`, `docs/`, `scripts/` directories.
- FastAPI skeleton with one `/ws` WebSocket endpoint.
- Frontend single-page HTML with Start/Stop button, camera preview, push-to-talk button, status indicator.
- HTTPS dev setup (`mkcert` for localhost, `ngrok` for cross-device testing).
- Frontend captures camera frames at 5 FPS and sends as JPEG over WebSocket.
- Backend receives frames and logs their size.
- Backend generates a hardcoded “hello” TTS phrase via gTTS every 3 seconds.
- Frontend plays received MP3 through speaker.
- Frontend captures push-to-talk audio and uploads as a single blob; backend writes it to a WAV file for inspection.
- Wake lock acquired on Start, released on Stop.
- README with setup steps. Pinned `requirements.txt`.

By Friday May 11, all moving parts connect end-to-end. Everything after this replaces stubs with real logic.

4.2 Sprint 2: Voice Pipeline (May 12 to May 18)

Goal: the user can speak commands and the system understands them.

Deliverables:

- Whisper integrated on the backend (`whisper-base`, English).
- Push-to-talk audio decoded server-side from WebM/Opus to PCM and transcribed.
- Command parser that recognizes the two task families and extracts target object or destination, with fuzzy matching against a curated phrase list.
- Clarification flow when intent is ambiguous (“which object did you say?”).

- TTS confirmation responses (“looking for your cup”, “navigating to the kitchen”).
- FSM with Idle, ListeningForCommand, ObjectAllocationActive (placeholder), NavigationActive (placeholder), ReturningToIdle.
- End-to-end test: user says “find my cup” and hears “looking for your cup” within 2 seconds.

4.3 Sprint 3: Object Allocation Task (May 19 to June 1, two weeks)

Goal: the full Object Allocation task works end-to-end.

Deliverables:

- YOLOv8n integration on the backend with COCO weights via the `ultralytics` package.
- Frame decoding and inference pipeline running at the rate frames arrive.
- Spatial reasoning module mapping bounding boxes to direction (left / center / right) and distance (near / medium / far) categories. Distance thresholds tuned empirically against a few reference objects.
- Temporal consistency check: target detected in 3 of last 5 frames before issuing guidance.
- Template-based guidance generator for Object Allocation.
- Throttling: emit new guidance only when region or distance category changes.
- Task completion logic: user confirmation (“got it”), explicit cancellation (“stop”), or 60-second timeout.
- Edge case handling: target never detected (after 30 seconds, prompt user to scan), multiple instances (guide toward most centered), object lost from view (use last-known phrasing).
- End-to-end test: blindfolded team member finds a cup placed in the test room using only audio guidance.

This sprint gets two weeks because it is the hardest single piece and it is the first task family that actually uses the vision pipeline.

4.4 Sprint 4: Navigation Task (June 2 to June 8)

Goal: the Navigation task works end-to-end with user-provided waypoints.

Deliverables:

- Navigation command parser that asks for the first landmark when a destination is given.
- Waypoint storage in task context (a list of user-provided strings).

- Landmark detection from the YOLOv8n output stream using available COCO furniture classes (chair, couch, dining table, refrigerator) as proxy landmarks.
- Template-based turn-by-turn guidance (“continue forward”, “turn slightly left”, “stop here”).
- Obstacle warning logic that preempts navigation cues when an object is detected close in the forward path.
- Waypoint-reached detection (target landmark visible and centered) triggers a prompt for the next waypoint.
- Completion: user confirmation (“I’m here”), cancellation, or timeout.
- End-to-end test: blindfolded team member navigates a simple route with two waypoints in the test room.

We accept that COCO does not have door/hallway/stairs classes. The task leans heavily on user-provided waypoints. If the COCO furniture proxies prove unusable in our test space, we narrow the test scenario rather than expand the model.

4.5 Sprint 5: Polish, Trials, and Report (June 9 to June 15)

Goal: the system is demo-ready, the report sections affected by the architecture change are updated, and we have measured numbers.

Deliverables:

- Error handling for the cases that have actually shown up: WebSocket drops (fall back to Idle), Whisper failures (clarification prompt), permission denial (user-visible message), camera unavailable (user-visible message).
- Performance measurement: log latency distributions, frame rate sustained, command recognition rate, end-to-end task success rate.
- Internal trials: each team member runs the system blindfolded for 5 Object Allocation trials and 5 Navigation trials while the other two facilitate. Results recorded.
- Report Section 4.2 rewritten for the browser-based architecture (Diaa).
- Report Section 4.3 updated to reflect the implemented software stack (split: Ahmad on vision, Omar on voice/FSM).
- Report Section 5 (Evaluation) populated with real numbers from our trials.
- Demo script written and rehearsed.
- Final README pass: clear setup steps, supported phones, known limitations.

5. Workload Distribution

After Sprint 1 (which we do together), each person leads one vertical area. We pair on integration points and review each other's pull requests. Every Friday we test together.

5.1 Diaa: Frontend, Audio Pipeline, and Integration

Owns the browser application end-to-end: media capture, WebSocket client, audio playback queueing, wake lock, UI shell, frame uploader, push-to-talk handling. Acts as **integration owner** — runs the Friday end-to-end smoke test, is the first call when something breaks across the client-server boundary.

Sprint-by-sprint: Sprint 1 sets up the frontend and HTTPS dev environment, builds the vertical slice on the client side. Sprint 2 implements the push-to-talk UI and clarification prompt flow. Sprint 3 implements the Object Allocation client UX and tests the full loop. Sprint 4 adds Navigation client flow (waypoint prompts, obstacle warning playback). Sprint 5 owns the audio queueing edge cases and the frontend portion of the report rewrite.

Risk areas Diaa watches: frame encoding cost on low-end phones, audio playback overlap, mic capture during TTS playback (mitigated by push-to-talk), Chrome differences between Android and iOS.

5.2 Ahmad: Vision Pipeline and Object Allocation Task

Owns visual perception: YOLOv8n integration, frame decoding, spatial reasoning (direction and distance categorization), temporal consistency, and the Object Allocation task manager and guidance generator. Lead for Sprint 3.

Sprint-by-sprint: Sprint 1 sets up the backend repo and dependencies for the vision stack, builds the frame-receiving stub. Sprint 2 helps Omar with backend integration. Sprint 3 leads — main focus. Sprint 4 contributes landmark detection and obstacle detection that Navigation depends on. Sprint 5 produces perception evaluation numbers (detection rate, position accuracy, guidance stability) for the report.

Risk areas Ahmad watches: YOLOv8n inference latency, COCO class limitations for indoor use, false positive guidance from transient detections, distance category thresholds (require empirical tuning).

5.3 Omar: Voice Pipeline, FSM, and Navigation Task

Owns speech and control logic: Whisper, audio chunking, command parser, FSM and task manager, gTTS, guidance generator infrastructure, and the Navigation task. Lead for Sprint 2 and Sprint 4.

Sprint-by-sprint: Sprint 1 sets up the FastAPI WebSocket scaffolding and the placeholder TTS round trip. Sprint 2 leads — main focus on voice. Sprint 3 contributes FSM hooks and guidance generator infrastructure. Sprint 4 leads — main focus on Navigation. Sprint 5 produces voice evaluation numbers (command recognition, intent accuracy, latency) for the report.

Risk areas Omar watches: Whisper accuracy on short commands, FSM edge cases (cancellation race conditions), gTTS latency interacting with the audio queue.

5.4 Shared Responsibilities

Some work is split across the team rather than owned by one person: the report re-write (Diaa: client section; Ahmad: perception section; Omar: voice and FSM section), trial design and execution (planned together, all three rotate roles as test subject and facilitator), code review (every PR gets one reviewer who is not the author), demo prep (rehearsed together), README updates (whoever changes a setup step updates the doc).

6. Technical Decisions Locked In

These are decided. We do not relitigate during implementation unless we hit a hard blocker.

- **Backend framework:** FastAPI with `uvicorn`.
- **WebSocket protocol:** binary frames for media, JSON text frames for control.
- **Frame format:** JPEG quality 70, capture resolution 640x480, server-side resize to 640x640 letterbox before YOLO. Target 5 FPS.
- **Audio capture:** push-to-talk only. Opus in WebM container (default `MediaRecorder` output on Chrome), decoded server-side to PCM for Whisper.
- **Audio playback:** MP3 from gTTS, played via `Audio` element with a queue.
- **ML models:** `ultralytics` YOLOv8n with default COCO weights; `openai-whisper` `whisper-base` (English); gTTS English voice.
- **Hosting (development):** local machines plus `ngrok` or Cloudflare Tunnel for HTTPS exposure to phones.
- **Hosting (demo):** a single VPS with a real domain and Let's Encrypt cert. Single concurrent session.
- **Repo:** monorepo. `main` is always demo-able. Feature branches `sprint-N-feature`. Every PR needs one reviewer.

7. Risks and Mitigations

The risks below are ones we expect to actually hit, ordered roughly by likelihood.

- **Latency feels bad on cellular.** *Mitigation:* throttling design means few messages, so each one being slow is more tolerable. Test on cellular early.
- **COCO is insufficient for navigation landmarks.** *Mitigation:* Sprint 4 leans on user-provided waypoints. Furniture as proxy landmarks. If results are unusable, narrow the test scenario rather than expand the model.
- **Whisper transcription errors on short commands.** *Mitigation:* clarification flow, fuzzy matching against a curated command-phrase list.
- **Distance category thresholds are wrong on first try.** *Mitigation:* allocate explicit time in Sprint 3 to tune thresholds against reference objects in our test room.
- **Sprint 3 (Object Allocation) overruns its two weeks.** *Mitigation:* if not on track by end of week one of Sprint 3, defer edge-case handling to Sprint 5 and move on. Do not eat into Sprint 4.
- **Mic and camera contention on low-end phones.** *Mitigation:* test on at least two phones early. Document minimum supported phones in README.
- **Permission re-prompts annoy the user every session.** *Mitigation:* documented as known limitation. Future native app fixes this.
- **Three-person coordination overhead.** *Mitigation:* Diaa is integration owner and runs Friday smoke tests. We pair on integration boundaries (frame upload contract, audio response contract, FSM event contract).

8. Definition of Done for v1

We consider Lumen v1 complete when all of the following hold:

- A user opens our URL on a Chrome browser (Android or iOS), grants permissions, and uses the system without further configuration.
- The system completes both Object Allocation and Navigation tasks end-to-end with verbal guidance only.
- A blindfolded team member completes a “find the cup” task in our test room with at least 60% success rate over 5 trials.
- A blindfolded team member navigates a simple two-waypoint route with at least 40% success rate over 5 trials.
- All code is in the monorepo, documented in the README, and runnable from a fresh clone.
- Report Sections 4.2, 4.3, and 5 reflect what we actually built and measured.

The success thresholds are deliberately lower than what we initially proposed. With 6.5 weeks and three people, hitting these is honest; promising more would be wishful.

9. Out of Scope (Tracked for Future Work)

We list these so we don't lose track of them but we do not commit to building any of them in this project: Arabic language support, native mobile application, edge inference (YOLO on-device), custom indoor dataset and fine-tuned model with architectural classes, Hand Pickup Assistance as a separate mode, voice activity detection (no push-to-talk), LLM-based guidance phrasing, multi-user concurrent sessions, persistent user preferences, Bluetooth earpiece integration, Safari and Firefox support, offline mode, formal user studies with blind participants, reconnection with task-state recovery.